

Real-Time Video Capture and VGA Processing System

Group Members

Group name: แกงฟักทอง

Name	Student Number
Nathawat Wattanarapeepong	6732010421
Piranat Wadlom	6732031621
Puntawit Masun	6732028821
Kritarat Moonmanee	6732001821

Abstract

This project implements a real-time video system on the Basys 3 FPGA. The FPGA receives image data from an OV7670 camera through a parallel camera interface, stores RGB565 pixel data in Block RAM, reads the stored frame using VGA 640x480 timing, applies a switch-selected image filter through `SW[1:0]`, and sends the final video signal to a VGA monitor through `vga_r`, `vga_g`, `vga_b`, `vga_hsync`, and `vga_vsync`.

The design is divided into clear hardware modules so that clock domains, memory access, communication protocol logic, and image processing can be handled separately. The main modules are `ov7670_config`, `sccb_master`, `ov7670_capture`, `frame_buffer`, `vga_controller`, `display_scaler`, `filter_engine`, and `top`.

Overall Design Block Diagram

OV7670 to VGA System Architecture

Configuration path, capture path, frame buffer, display mapping, and filter selection

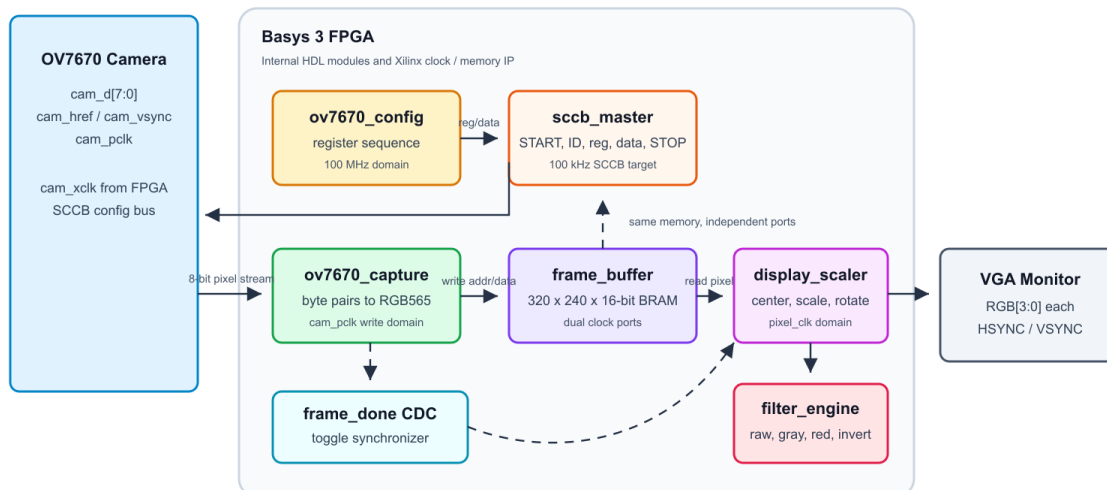


Figure 1. Overall system architecture. The OV7670 camera sends image data into the FPGA, *ov7670_capture* writes pixels into BRAM, the VGA pipeline reads pixels from BRAM for display, and a separate configuration path initializes the camera through SCCB.

The SCCB line in the block diagram represents the `cam_scl` and `cam_sda` signals used by `sccb_master` to write OV7670 registers during system startup.

The main video data path is:

```
OV7670 camera -> ov7670_capture -> frame_buffer -> display_scaler -> filter_engine -> VGA monitor
```

The camera configuration path is separate from the video data path:

```
ov7670_config -> sccb_master -> OV7670 SCCB pins
```

System Specification

Item	Implementation in this project
FPGA board	Digilent Basys 3, Artix-7
Camera	OV7670 parallel camera module
Camera data format	RGB565, two 8-bit transfers per pixel
Stored frame size	320 x 240 pixels
Frame buffer format	16-bit RGB565
Frame buffer size	$320 \times 240 \times 16 = 1,228,800$ bits
VGA timing	640 x 480 visible area, 800 x 525 total timing
VGA pixel clock	25 MHz
Camera master clock	25 MHz <code>cam_xclk</code> generated by FPGA
SCCB clock target	100 kHz
User controls	<code>SW[1:0]</code> selects filter mode
Filters	raw, grayscale, red-only, inverted

Clock Domains

Clock Domains and Synchronization

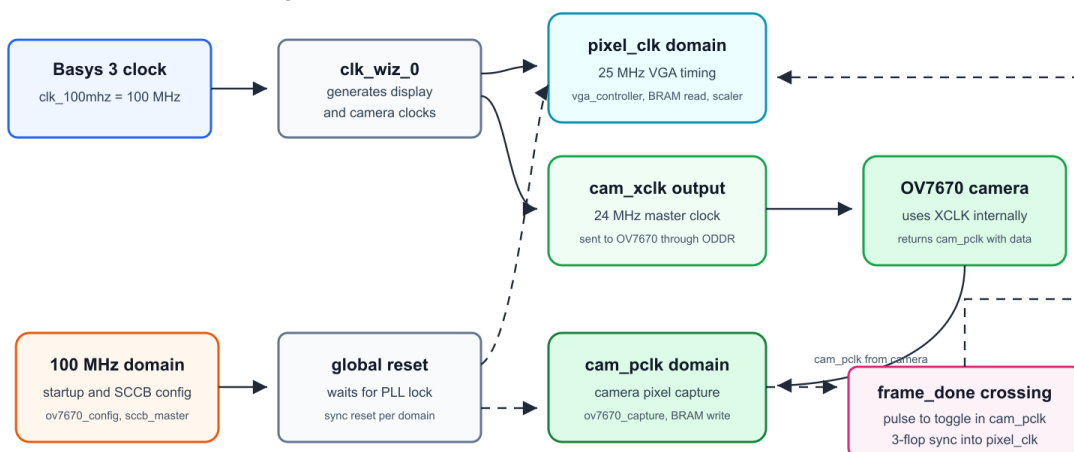


Figure 2. Clock domains in the system. The design uses a 100 MHz domain for configuration, a 25 MHz domain for VGA/display logic, a 25 MHz clock sent to the camera, and `cam_pclk` returned by the camera to time incoming image data.

The design cannot use a single clock for every block because each external interface has different timing requirements:

Clock	Source	Used by	Purpose
<code>clk_100mhz</code>	Basys 3 oscillator	<code>ov7670_config</code> , <code>sccb_master</code> , reset logic	Startup control and SCCB register writes
<code>pixel_clk</code>	<code>clk_wiz_0</code>	<code>vga_controller</code> , <code>frame_buffer</code> read port, <code>display_scaler</code>	VGA timing and display read pipeline
<code>cam_xclk</code>	<code>clk_wiz_0</code> + ODDR	OV7670 camera	Master clock supplied to the camera
<code>cam_pclk</code>	OV7670 camera	<code>ov7670_capture</code> , <code>frame_buffer</code> write port	Sampling clock for incoming camera data

The `frame_done` signal is generated in the `cam_pclk` domain but is consumed in the `pixel_clk` domain. To cross this clock boundary safely, `top.v` converts the pulse into a toggle, synchronizes it through three flip-flops, and then reconstructs a one-cycle pulse in the display domain. This pulse sets `frame_valid`, which prevents the VGA pipeline from showing startup garbage before the first complete frame is captured.

Design Decision

1. Module Architecture

The design is split into small modules based on real hardware responsibilities. This separation keeps unrelated timing, protocol, memory, and image-processing logic from being mixed together.

Module	Reason for separation
<code>ov7670_config</code>	Stores the register sequence that initializes the camera, so camera settings can be changed without modifying the bit-level SCCB master
<code>sccb_master</code>	Handles SCCB timing, including START, byte transfer phases, don't-care bits, and STOP
<code>ov7670_capture</code>	Runs in the camera clock domain and focuses on byte pairing, cropping, downsampling, and BRAM writes
<code>frame_buffer</code>	Wraps the Xilinx BRAM IP and provides a behavioral model for simulation
<code>vga_controller</code>	Generates VGA timing counters and sync pulses
<code>display_scaler</code>	Maps VGA screen coordinates to frame-buffer addresses
<code>filter_engine</code>	Performs combinational image processing selected by slide switches
<code>top</code>	Integrates modules, clocks, resets, clock-domain crossing, and FPGA I/O

This modular architecture makes debugging easier because the major blocks can be tested independently. The Cocotb testbenches verify the filter logic, VGA timing, camera capture behavior, and SCCB master behavior separately before the full system is integrated.

2. Communication Protocol

The OV7670 camera must be configured through SCCB, an I2C-like serial control bus. The design therefore uses `sccb_master` to generate a write transaction with this sequence:

```
START -> device write byte -> register address -> register data -> STOP
```

Camera Startup: Register Sequencer and SCCB Transaction

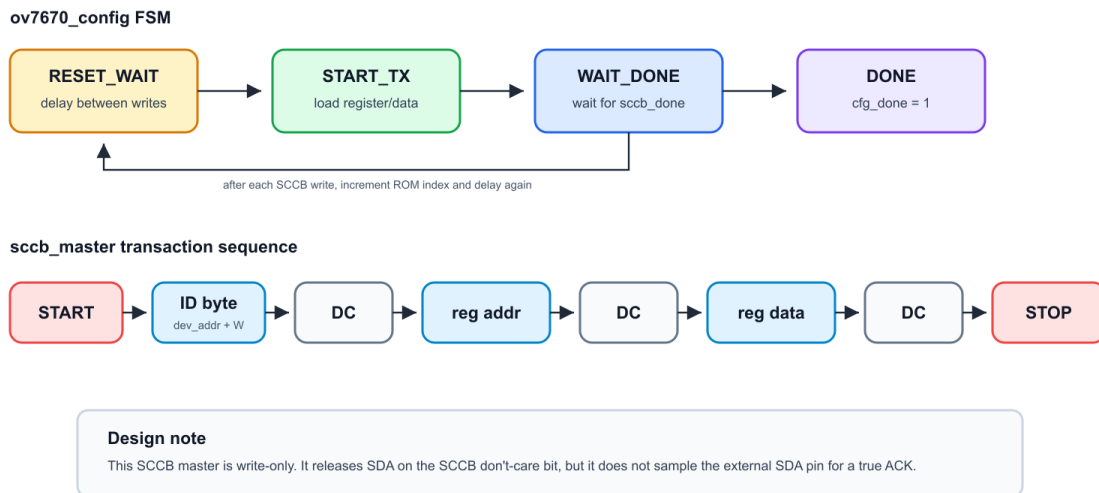


Figure 3. `ov7670_config` selects register/data pairs from its ROM-like case table, while `sccb_master` converts each pair into timing on `cam_scl` and `cam_sda`.

This project uses a write-only SCCB master because the camera only needs register writes during startup for normal operation. Reading registers back is not required for the implemented video pipeline.

3. Clock and Data Transfer Rate

VGA 640x480 output requires strict timing, so the design generates a 25 MHz `pixel_clk` where each clock corresponds to one VGA pixel slot. The OV7670 camera receives a 25 MHz `cam_xclk` from the FPGA and then returns its own `cam_pclk` to indicate when camera data is valid.

The true dual-port BRAM allows the write side and read side to use different clocks:

```
Port A: cam_pclk -> write camera pixels
Port B: pixel_clk -> read pixels for VGA
```

4. Resource Utilization

Storing a full 640 x 480 RGB565 frame would require:

```
640 x 480 x 16 = 4,915,200 bits
```

This is larger than the available BRAM capacity on the Basys 3. Therefore, this design stores a 320 x 240 RGB565 frame instead:

```
320 x 240 x 16 = 1,228,800 bits
```

This size fits the Artix-7 Block RAM resources on the Basys 3 while still providing enough image data for VGA display after coordinate mapping in `display_scaler`.

5. Display Mapping

The design does not use simple 2x pixel doubling. Instead, it uses the actual mapping implemented in `display_scaler.v`: the image is centered, scaled, and rotated before being displayed.

Display Mapping, Rotation, and Filter Pipeline

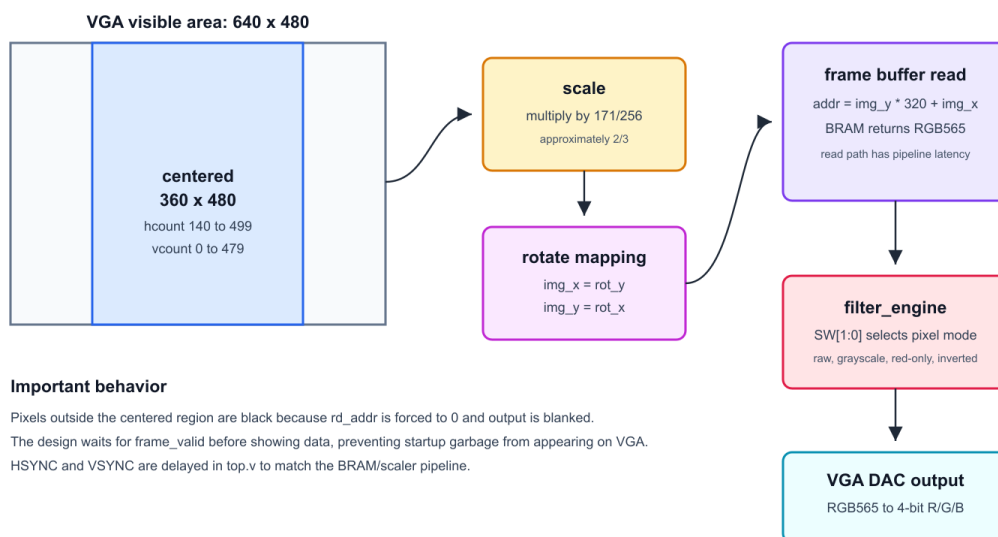


Figure 4. `display_scaler` selects a centered 360 x 480 region on the VGA screen, scales the coordinates by 171/256, rotates the coordinate mapping, reads BRAM, and sends the pixel into the filter stage.

This mapping is needed because the camera orientation used in this build does not directly match the desired VGA display orientation. The module swaps the coordinate axes:

```
img_x = rot_y;  
img_y = rot_x;
```

It then computes the frame-buffer address:

```
addr = img_y * 320 + img_x;
```

Implementation Detail

Module: top

Source: `hdl_sources/top.v`

`top` is the main integration module. It connects all major blocks and performs the following tasks:

- Receives `clk_100mhz`, reset, camera pins, and switches
- Generates `pixel_clk` and `cam_xclk` through `clk_wiz_0`
- Drives `cam_xclk` out through an ODDR
- Holds the system in reset until the PLL reports locked
- Connects the SCCB camera-configuration path
- Connects camera capture to the frame buffer
- Synchronizes `frame_done` from `cam_pclk` into `pixel_clk`
- Delays `hsync` and `vsync` to match the BRAM/display pipeline latency

The most important logic in this module is the clock-domain crossing for `frame_done`:

```
always @(posedge cam_pclk) begin
    if (cam_rst) frame_done_toggle <= 1'b0;
    else if (frame_done) frame_done_toggle <= ~frame_done_toggle;
end

always @(posedge pixel_clk) begin
    if (pix_rst) fd_sync <= 3'b0;
    else fd_sync <= {fd_sync[1:0], frame_done_toggle};
end

wire frame_done_pix = fd_sync[2] ^ fd_sync[1];
```

Instead of sending a one-cycle pulse directly across clock domains, the design turns the pulse into a toggle. The `pixel_clk` domain then detects a toggle change to recreate a one-cycle pulse. This is safer because a short pulse could be missed if sampled directly by another unrelated clock.

Module: ov7670_config

Source: `hdl_sources/ov7670_config.v`

`ov7670_config` is the OV7670 register-write sequencer. It stores {register address, register data} pairs in a case (`rom_idx`) table and sends one pair at a time to `sccb_master`.

The main FSM has four states:

State	Meaning
<code>S_RESET_WAIT</code>	Waits before starting the next register write
<code>S_START_TX</code>	Loads <code>reg_addr</code> , <code>reg_data</code> , and pulses <code>sccb_start</code>
<code>S_WAIT_DONE</code>	Waits for <code>sccb_done</code> from the SCCB master
<code>S_DONE</code>	Holds <code>cfg_done = 1</code> after all register writes are complete

Example logic:

```
if (current_reg == END_MARKER) begin
    cfg_done <= 1'b1;
    state    <= S_DONE;
end else begin
    reg_addr  <= current_reg[15:8];
    reg_data  <= current_reg[7:0];
    sccb_start <= 1'b1;
    state     <= S_WAIT_DONE;
end
```

The configuration values include soft reset, RGB565 output format, clock/scaling registers, sensor window settings, color matrix settings, exposure, gamma, saturation, contrast, and orientation.

Module: `sccb_master`

Source: `hdl_sources/sccb_master.v`

`sccb_master` generates the bit-level SCCB transaction. It uses `CLK_FREQ` and `SCCB_FREQ` to derive an internal tick:

```
localparam CLK_DIV = CLK_FREQ / (SCCB_FREQ * 4);
```

Each SCL period is divided into four phases so that SDA can change while SCL is low and remain stable while SCL is high.

The transaction sequence is:

```
ST_START
ST_ID      -> send {dev_addr, write_bit}
ST_DC_ID
ST_REG     -> send register address
ST_DC_REG
ST_DATA    -> send register data
ST_DC_DATA
ST_STOP
ST_DONE
```

Important note: this module does not sample the external `cam_sda` pin to check for a real ACK. The output port `sda` is an internal drive signal, and `top.v` converts it to open-drain behavior:

```
assign cam_sda = sda_out ? 1'bz : 1'b0;
```

Therefore, `ack_err` is only a legacy diagnostic output, not a true ACK result from the camera.

Module: `ov7670_capture`

Source: `hdl_sources/ov7670_capture.v`

`ov7670_capture` receives bytes from the camera using `cam_pclk` and combines every two bytes into one RGB565 pixel.

Camera Capture and Frame Buffer Addressing



Address formula used in RTL

Formula	Implemented as
$wr_addr = ((v_cnt - 4) / 2) * 320 + ((h_cnt - 4) / 2)$	$row * 256 + row * 64 + column, \text{ because } 320 = 256 + 64$

Why only every other pixel?

The camera is configured for VGA-size timing, but the frame buffer stores 320 x 240 pixels.
Keeping even columns and even rows performs simple 2:1 downsampling, which fits inside Basys 3 BRAM.

Example row after edge guard



Figure 5. Camera capture flow. The module combines camera bytes into RGB565 pixels, skips unstable image edges, downsamples by keeping every other pixel/row, and computes the BRAM write address.

The capture process works as follows:

1. If `cam_vsync` is high, the frame counters are reset.
2. If `cam_href` is high, the module is inside an active camera row.
3. The first byte is stored in `first_byte`.
4. The second byte is combined with it as `{first_byte, cam_d}`.
5. The first four rows/columns are skipped as an edge guard.
6. Only pixels where `h_cnt` and `v_cnt` are even are written.
7. The write address is computed in row-major order.

Address formula:

```
wr_addr = ((v_cnt - 4) / 2) * 320 + ((h_cnt - 4) / 2);
```

The RTL uses shifts to implement `row * 320`:

```
row * 320 = (row << 8) + (row << 6)
```

This works because $320 = 256 + 64$.

Module: `frame_buffer`

Source: `hdl_sources/frame_buffer.v`

`frame_buffer` wraps the 320 x 240 x 16-bit memory used between the camera-capture side and the VGA-display side.

In simulation, it uses a behavioral memory:

```
reg [15:0] mem [0:76799];
```

In synthesis, it instantiates the Xilinx `blk_mem_gen_0` IP:

```
blk_mem_gen_0 u_bram (  
    .clka    (clka),  
    .wea     (wea),  
    .addra   (addra),  
    .dina    (dina),  
    .clkb    (clkb),  
    .addrb   (addrb),  
    .doutb   (doutb)  
);
```

The true dual-port BRAM is useful because the camera writes pixels using `cam_pclk` while the VGA pipeline reads pixels using `pixel_clk`. This allows the two sides to operate at the same time without stalling each other.

Module: vga_controller

Source: hdl_sources/vga_controller.v

`vga_controller` generates the counters for VGA 640 x 480 timing. One complete frame uses these timing values:

Timing	Value
Horizontal visible	640
Horizontal front porch	16
Horizontal sync	96
Horizontal back porch	48
Horizontal total	800
Vertical visible	480
Vertical front porch	10
Vertical sync	2
Vertical back porch	33
Vertical total	525

`hcount` counts from 0 to 799 and then wraps back to 0. `vcount` increments at the end of each horizontal line and counts from 0 to 524.

The sync signals are active-low:

```
assign hsync = ~( (hcount >= 656) && (hcount < 752) );  
assign vsync = ~( (vcount >= 490) && (vcount < 492) );
```

The `active` signal is high only inside the visible region:

```
assign active = (hcount < 640) && (vcount < 480);
```

Module: display_scaler

Source: hdl_sources/display_scaler.v

`display_scaler` runs in the `pixel_clk` domain. It converts the current VGA position (`hcount`, `vcount`) into the frame-buffer address that should be read.

Important steps:

8. Select only the centered horizontal region `hcount = 140..499`.
9. Scale `screen_x` and `vcount` by 171/256.
10. Swap axes to correct the camera orientation.
11. Compute `rd_addr`.
12. Send `rd_data` into `filter_engine`.
13. Convert RGB565 into 4-bit R/G/B values for the VGA DAC.

Blanking behavior:

```
if (!active_d2 || !frame_valid) begin
    vga_r <= 4'd0;
    vga_g <= 4'd0;
    vga_b <= 4'd0;
end
```

Therefore, the display is black before the first complete frame is captured and also black outside the selected active display area.

Module: filter_engine

Source: hdl_sources/filter_engine.v

`filter_engine` is a combinational module that receives one RGB565 pixel and outputs a modified pixel based on `sw[1:0]`.

RGB565 Filter Modes Selected by SW[1:0]

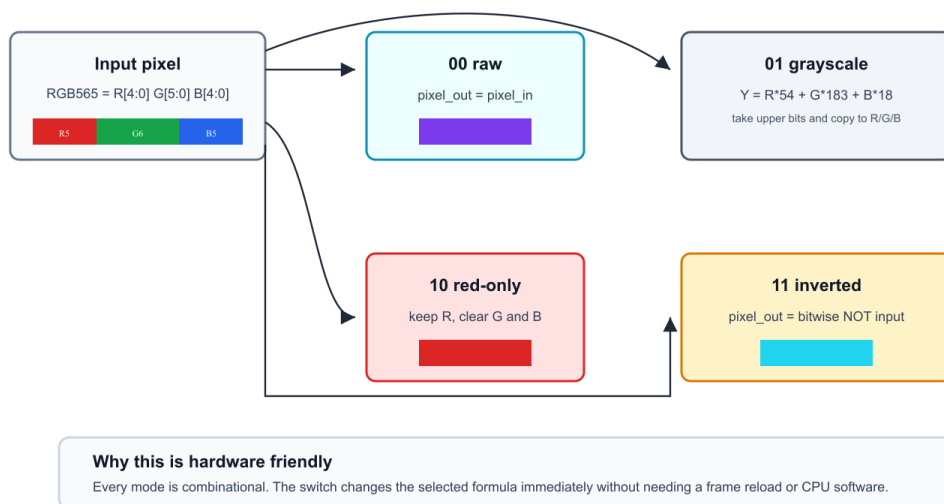


Figure 6. Filter modes selected by `SW[1:0]`.

SW[1:0]	Mode	Behavior
00	Raw	Pass the input pixel through unchanged
01	Grayscale	Compute luminance and copy it into R/G/B
10	Red-only	Keep only the red channel
11	Inverted	Invert every bit of the RGB565 pixel

Grayscale formula in the RTL:

```
wire [13:0] y_scaled = (r5 * 14'd54) + (g6 * 14'd183) + (b5 * 14'd18);  
wire [5:0] y_6bit = y_scaled[13:8];  
wire [4:0] y_5bit = y_6bit[5:1];
```

The green channel has the highest weight because human vision is more sensitive to green brightness than to red or blue brightness.

Constraints: `constraints.xdc`

Source: `hdl_sources/constraints.xdc`

The constraints file maps logical top-level ports to physical FPGA pins:

Interface	Pins / ports
System clock	<code>clk_100mhz</code> on W5, 100 MHz
Reset	<code>rst</code> on U18
Camera data	<code>cam_d[0]..cam_d[7]</code>
Camera control	<code>cam_href</code> , <code>cam_pclk</code> , <code>cam_vsync</code> , <code>cam_xclk</code> , <code>cam_scl</code> , <code>cam_sda</code>
VGA output	<code>vga_r[3:0]</code> , <code>vga_g[3:0]</code> , <code>vga_b[3:0]</code> , <code>vga_hsync</code> , <code>vga_vsync</code>
Switches	<code>sw[1:0]</code>
LEDs	<code>led[2:0]</code>

It also declares clock constraints and tells Vivado that `clk_100mhz`, `pixel_clk`, and `cam_pclk` are asynchronous clock groups.

Simulation and Testbenches

Testbench	Module under test	Main checks	Result
<code>sim/test_filter_engine.py</code>	<code>filter_engine</code>	raw, grayscale, red-only, invert, switch changes	8 passed
<code>sim/test_vga_controller.py</code>	<code>vga_controller</code>	reset, counter wrap, active area, sync pulse timing	8 passed
<code>sim/test_ov7670_capture.py</code>	<code>ov7670_capture</code>	reset, VSYNC frame_done, edge guard, address math, <code>wr_en</code> pulse	6 passed
<code>sim/test_sccb_master.py</code>	<code>sccb_master</code>	idle, start, done pulse, busy, SCL/SDA behavior, back-to-back transfer	7 passed

Waveform: filter_engine

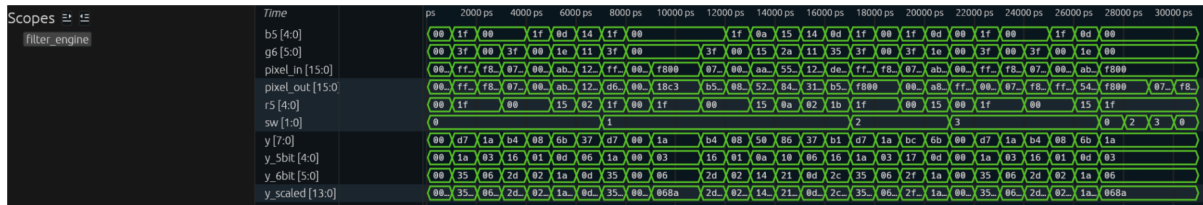


Figure 8. filter_engine waveform. When sw changes, the output changes immediately according to the selected mode because the module is combinational.

Waveform: vga_controller

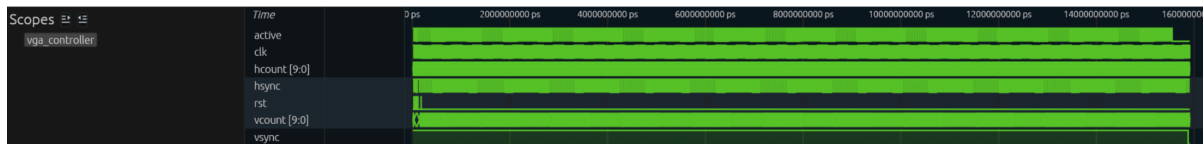


Figure 9. vga_controller waveform showing VGA counters and sync timing.

Waveform: ov7670_capture

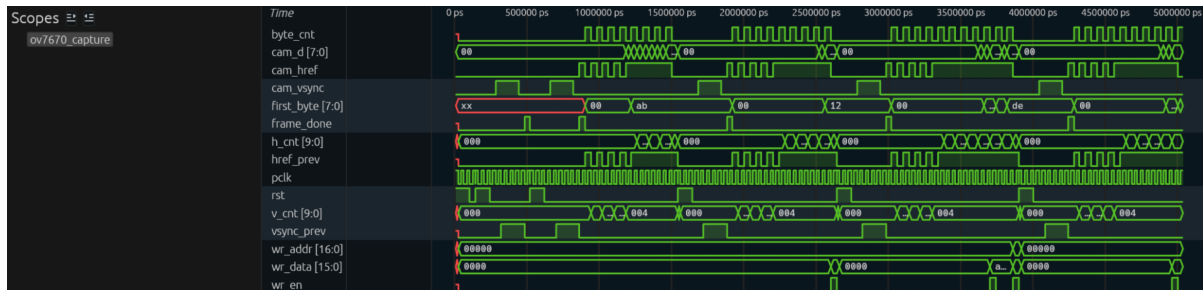


Figure 10. ov7670_capture waveform showing byte pairing, wr_en pulses, and address calculation for pixels that pass the edge guard and downsampling conditions.

Waveform: sccb_master

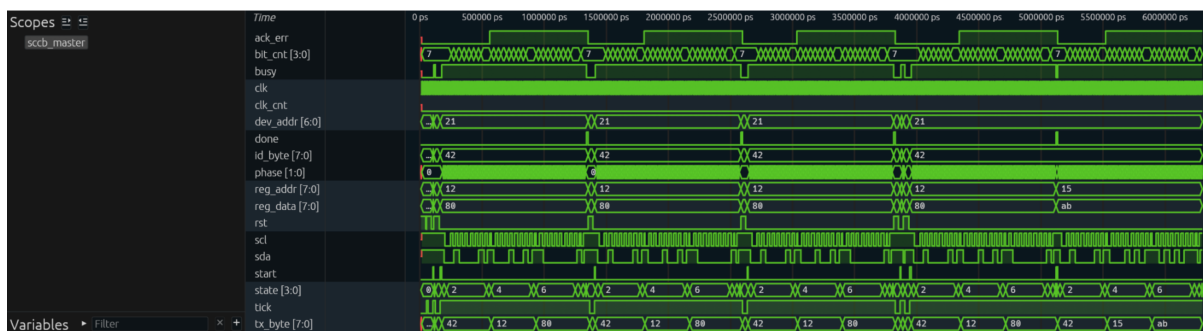


Figure 11. sccb_master waveform showing the start, busy, scl, sda, and done signals during an SCCB transaction.

Challenges Faced

1. OV7670 documentation and register setup

The OV7670 has many configuration registers, and parts of the documentation are difficult to interpret. A key challenge was choosing register values that make the camera output RGB565 data with timing suitable for this FPGA design.

2. Multiple clock domains

The system uses `clk_100mhz`, `pixel_clk`, `cam_xclk`, and `cam_pclk`. Sending signals directly between unrelated clock domains can cause metastability or missed pulses, so the design uses reset synchronization and a toggle synchronizer for `frame_done`.

3. BRAM capacity

The Basys 3 has limited Block RAM, so it cannot store a full 640 x 480 x 16-bit frame. The design solves this by storing a 320 x 240 RGB565 frame and mapping it into the VGA visible area.

4. VGA timing and pipeline alignment

The VGA monitor requires accurate sync timing. Since BRAM reads and the display pipeline introduce latency, `top.v` delays `hsync` and `vsync` so that the sync outputs stay aligned with the displayed pixel data.

5. Debugging real-time video

Real-time video is harder to debug than simple logic because small errors in clocking, address calculation, or sync timing can produce a black screen or distorted image. The project therefore uses module-level testbenches and waveform inspection to confirm behavior before full-system integration.

AI Usage Declaration

1. Used to assist in designing the data path.
2. Used to assist in implementing the pipeline as planned.
3. Used to assist in debugging.
4. Used to assist in writing testbenches.
5. Used to assist in writing comments to explain various parts of the code.
6. Used to assist in creating diagrams for the report and writing the report.